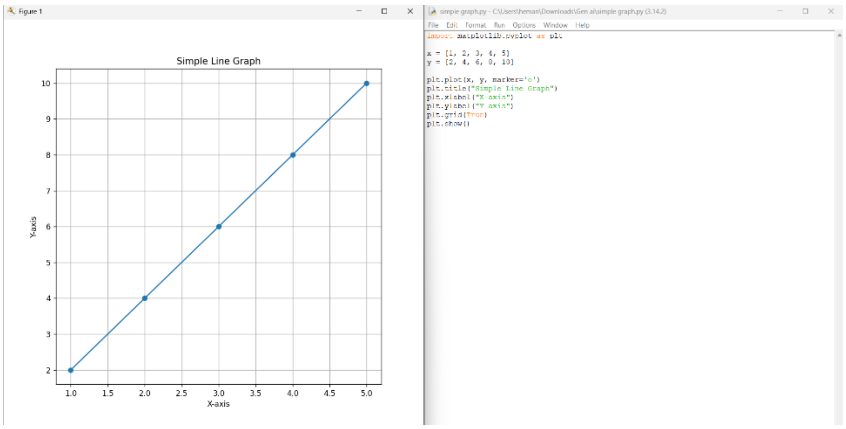
1.Program

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]

y = [2, 4, 6, 8, 10]

plt.plot(x, y, marker='o')

plt.title("Simple Line Graph")

plt.xlabel("X-axis")

plt.ylabel("Y-axis")

plt.grid(True)

plt.show()
```

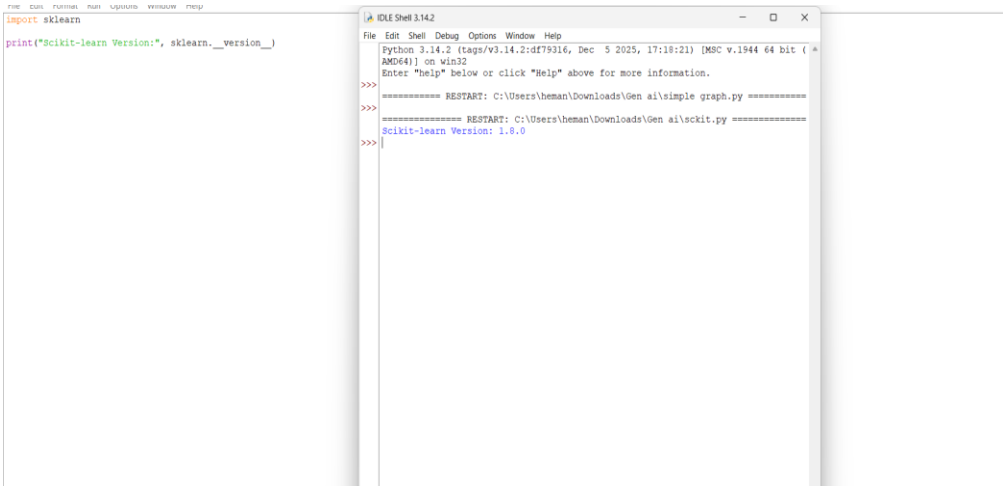
Output



2.Program

```
import sklearn

print("Scikit-learn Version:", sklearn.__version__)
```

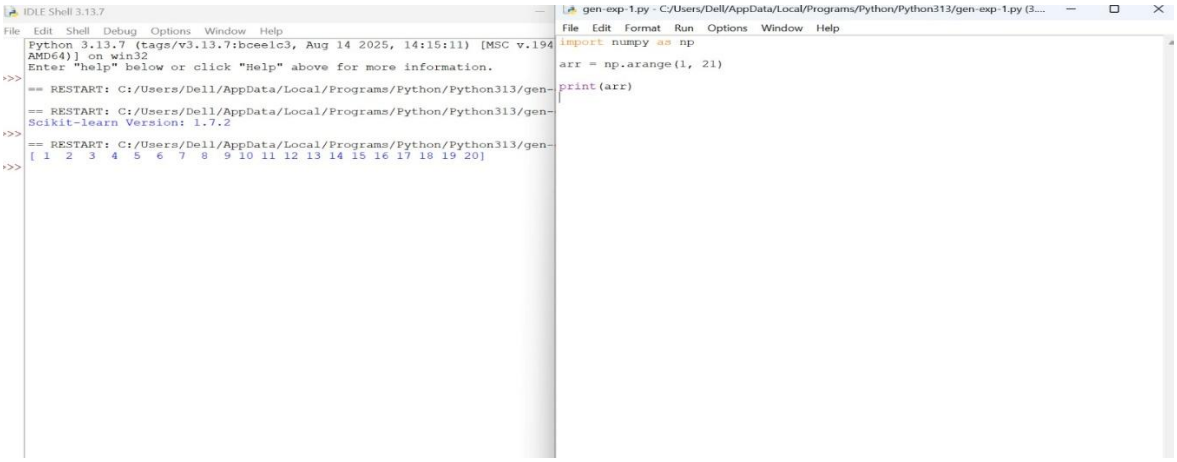
OUTPUT



3.PROGRAM

```
import pandas as pd

data = {

    "Student": ["Alice", "Bob", "Charlie", "David"],

    "Grade": ["A", "B+", "A+", "B"]

}

df = pd.DataFrame(data)

print(df)
```

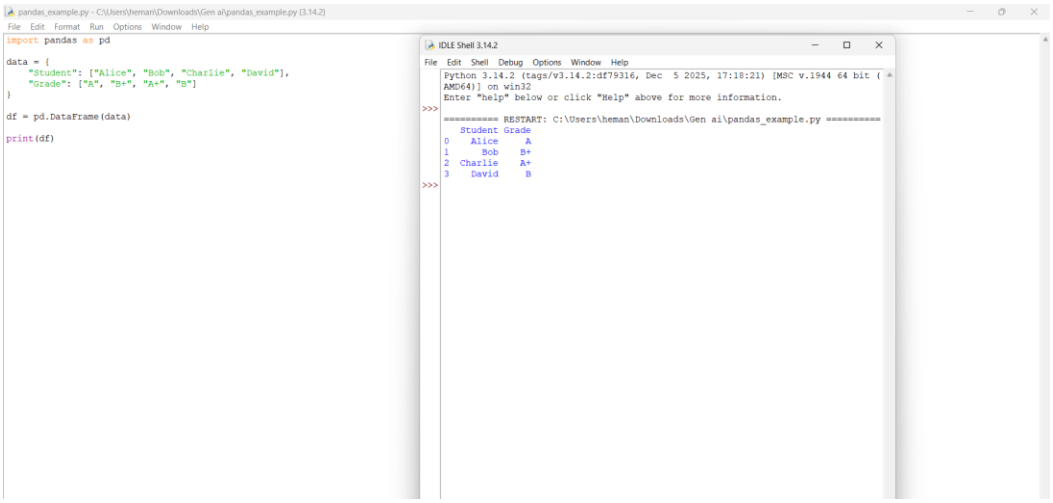
OUTPUT



4.PROGRAM

```
import pandas as pd

data = {

 "Name": ["Sai", "Rahul", "Anu", "Priya"],

 "Grade": [90, 85, 95, 88] }

 df = pd.DataFrame(data)

 print(df)
```

OUTPUT

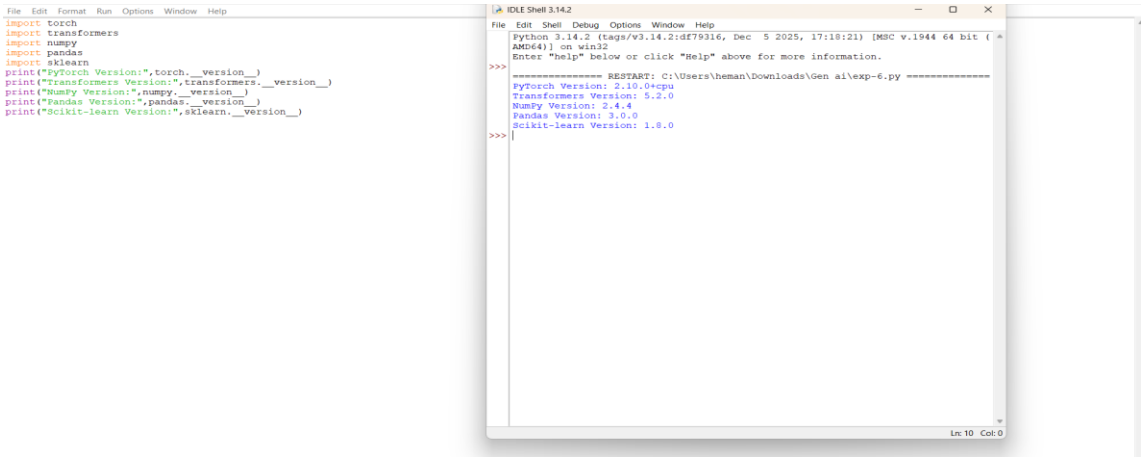


5.PROGRAM

```
import torch

import transformers

import numpy

import pandas

import sklearn

print("PyTorch Version:", torch.__version__)

print("Transformers Version:", transformers.__version__)

print("NumPy Version:", numpy.__version__)

print("Pandas Version:", pandas.__version__)
```

print("Scikit-learn Version:", sklearn.__version__)

OUTPUT



6.PROGRAM

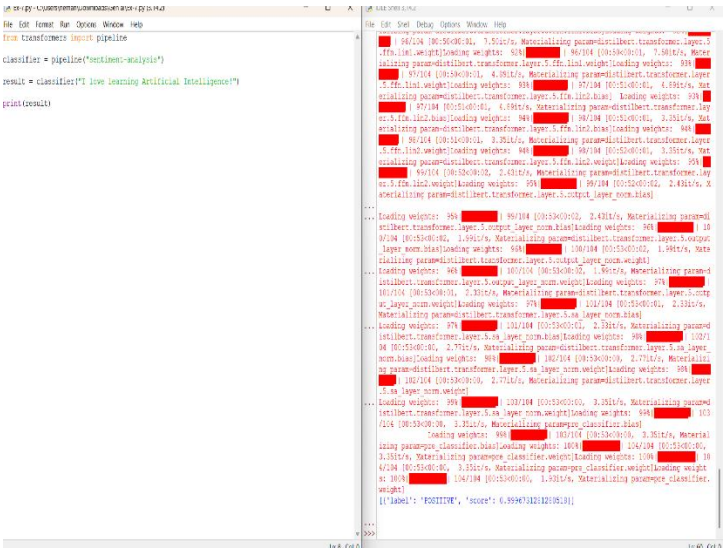
```
import torch

a = torch.tensor([1, 2, 3])

b = torch.tensor([4, 5, 6])

print("Tensor A:", a)

print("Tensor B:", b)

print("Addition:", a + b)

print("Subtraction:", a - b)

print("Multiplication:", a * b)

print("Division:", a / b)

print("Dot Product:", torch.dot(a, b))
```

OUTPUT

7 PROGRAM

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

result = classifier("I love learning Artificial Intelligence!")

print(result)
```

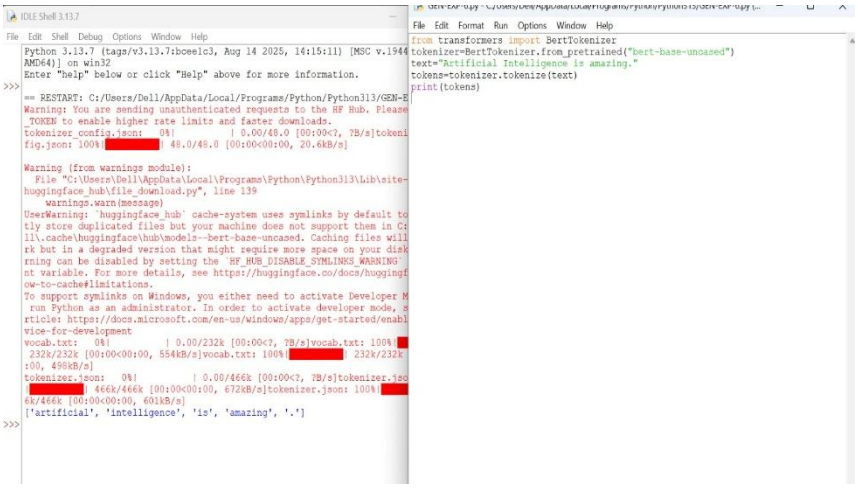
OUTPUT



8.PROGRAM

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

text = "Artificial Intelligence is amazing."

tokens = tokenizer.tokenize(text)

print(tokens)
```

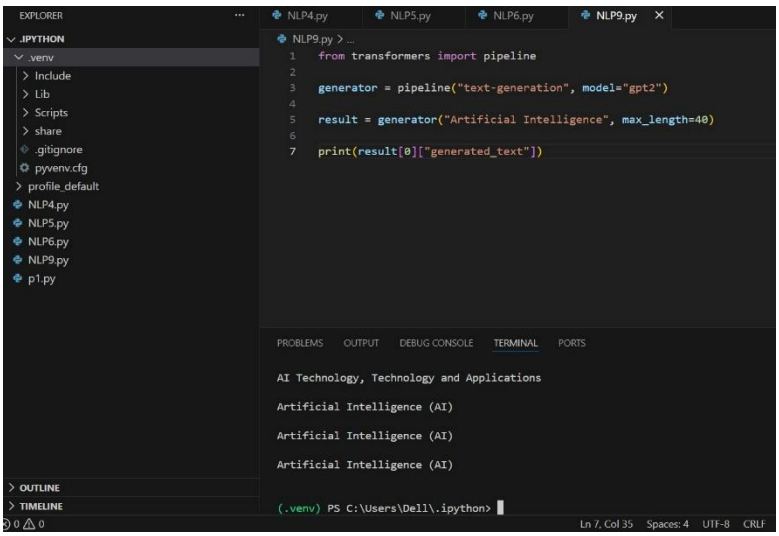
OUTPUT



9.PROGRAM

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

result = generator("Artificial Intelligence", max_length=50)

print(result[0]["generated_text"])
```

OUTPUT



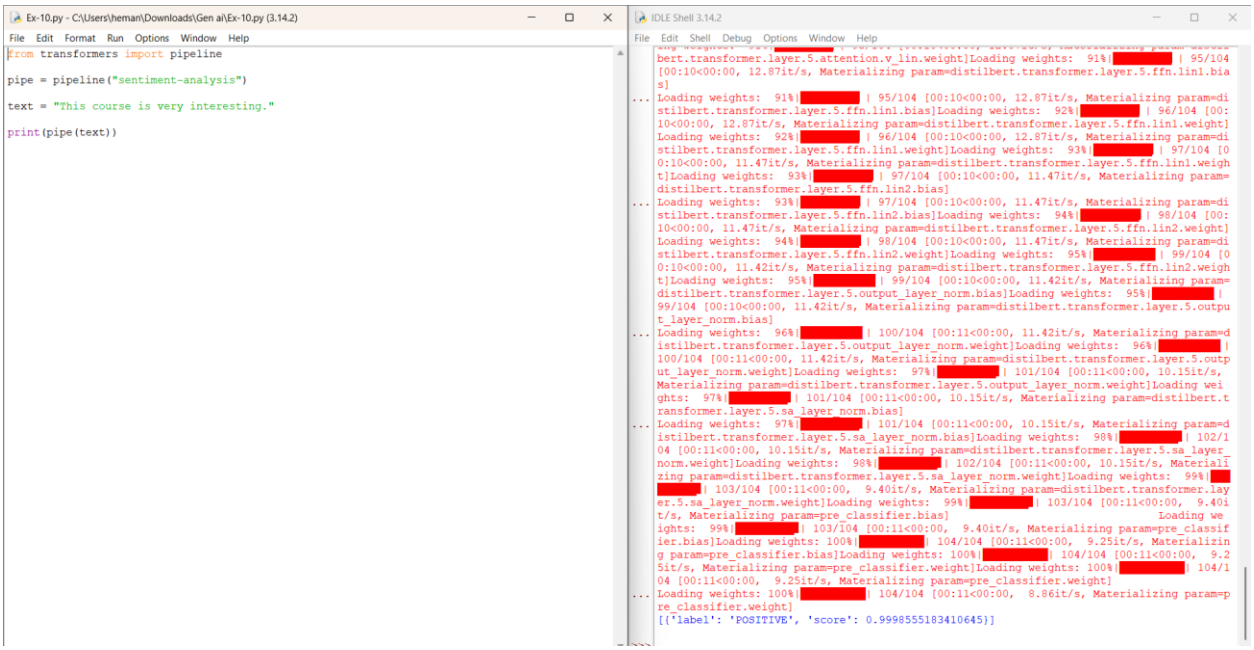
10 PROGRAM

```
from transformers import pipeline

pipe = pipeline("sentiment-analysis")
```

text = "This course is very interesting."

print(pipe(text))

OUTPUT



11 PROGRAM

from transformers import BertTokenizer

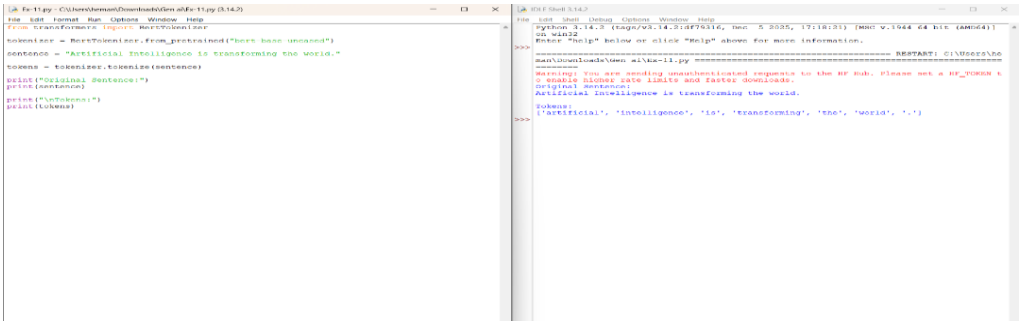
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

sentence = "Machine learning is the future."

tokens = tokenizer.tokenize(sentence)

print(tokens)

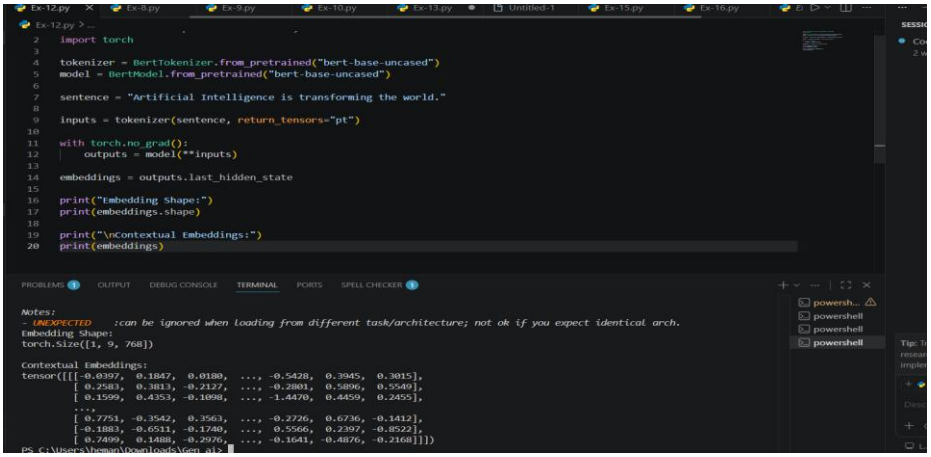
OUTPUT



12 PROGRAM

```
from transformers import BertTokenizer, BertModel
import torch
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertModel.from_pretrained("bert-base-uncased")
sentence = "Artificial Intelligence is changing the world."
inputs = tokenizer(sentence, return_tensors="pt")
with torch.no_grad():
    outputs = model(**inputs)
embeddings = outputs.last_hidden_state
print(embeddings.shape)
print(embeddings)
```

OUTPUT



13 PROGRAM

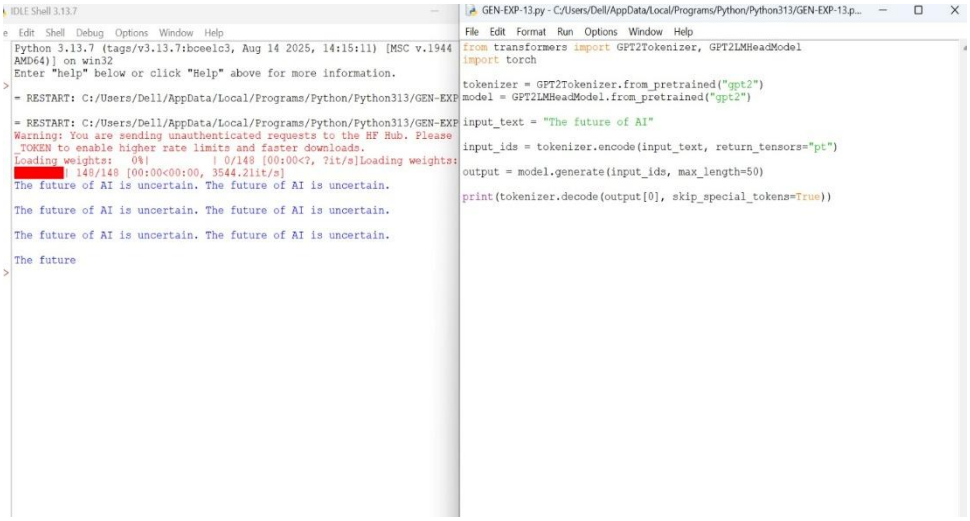
```
from transformers import GPT2Tokenizer, GPT2LMHeadModel
import torch
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
model = GPT2LMHeadModel.from_pretrained("gpt2")
prompt = "The future of AI"
```

inputs = tokenizer.encode(prompt, return_tensors="pt")

outputs = model.generate(inputs, max_length=50)

print(tokenizer.decode(outputs[0], skip_special_tokens=True))

OUTPUT



14 PROGRAM

from transformers import pipeline

qa = pipeline("question-answering")

context = """

Python is a popular programming language developed by Guido van Rossum.

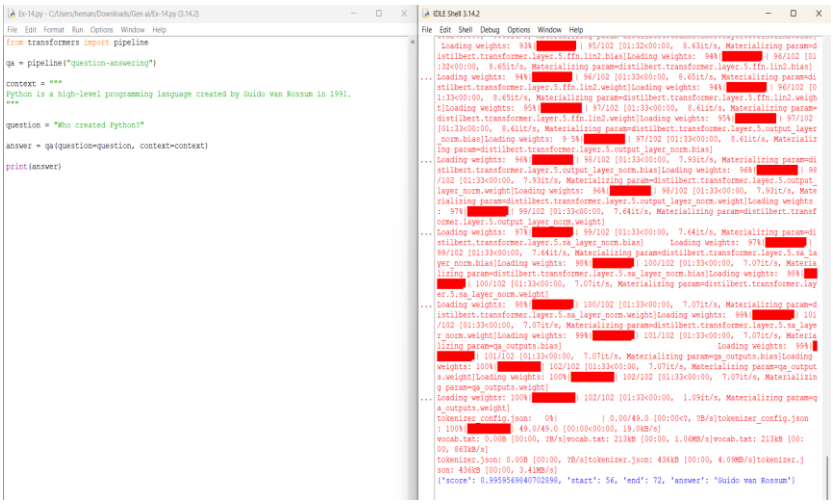
"""

question = "Who developed Python?"

result = qa(question=question, context=context)

print(result)

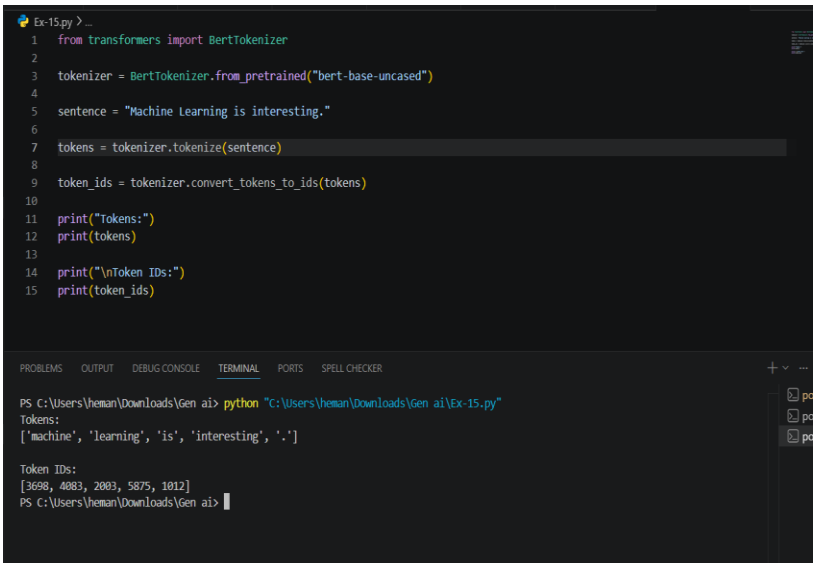
OUTPUT



15 PROGRAM

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

sentence = "Deep learning is powerful."

tokens = tokenizer.tokenize(sentence)

token_ids = tokenizer.convert_tokens_to_ids(tokens)

print("Tokens:", tokens)

print("Token IDs:", token_ids)
```

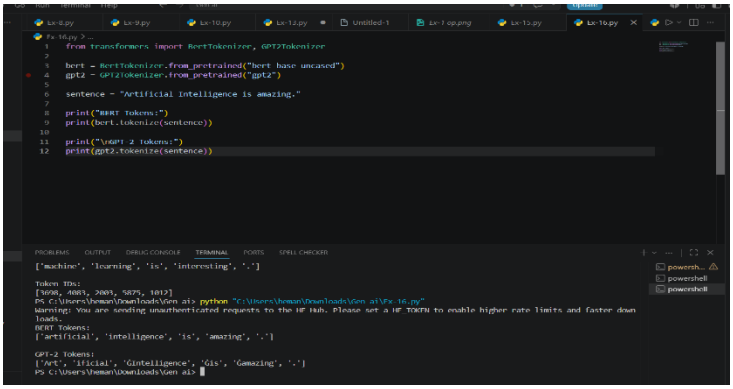
OUTPUT



16 PROGRAM

```
from transformers import BertTokenizer, GPT2Tokenizer

bert = BertTokenizer.from_pretrained("bert-base-uncased")

gpt2 = GPT2Tokenizer.from_pretrained("gpt2")

text = "Artificial Intelligence is amazing."

print("BERT Tokens")

print(bert.tokenize(text))

print("\nGPT-2 Tokens")

print(gpt2.tokenize(text))
```

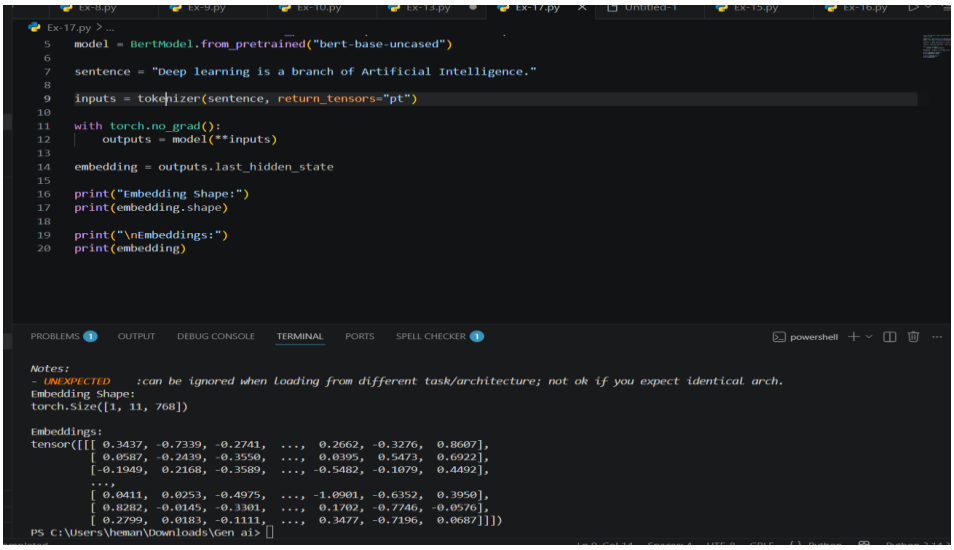
OUTPUT



17 PROGRAM

```
from transformers import BertTokenizer, BertModel

import torch

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

model = BertModel.from_pretrained("bert-base-uncased")

text = "Natural Language Processing is fascinating."

inputs = tokenizer(text, return_tensors="pt")

with torch.no_grad():

    outputs = model(**inputs)

print(outputs.last_hidden_state)
```

OUTPUT

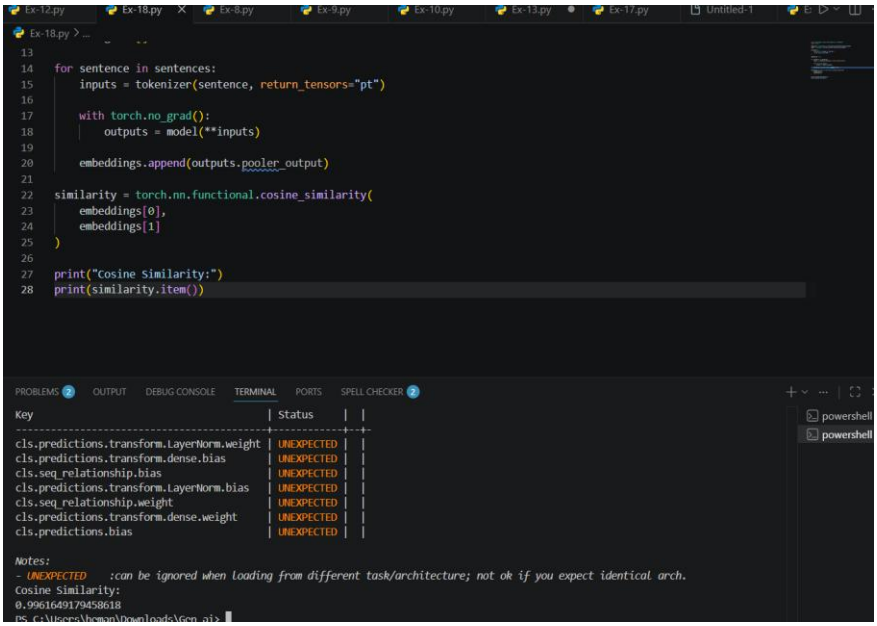


18 PROGRAM

```
from transformers import BertTokenizer, BertModel
import torch
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertModel.from_pretrained("bert-base-uncased")
sentences = [
    "The cat is sitting on the mat.",
    "A cat sits on the mat."
]
embeddings = []
for sentence in sentences:
    inputs = tokenizer(sentence, return_tensors="pt")
    with torch.no_grad():
        output = model(**inputs)
    embeddings.append(output.pooler_output)
similarity = torch.nn.functional.cosine_similarity(
```

```
    embeddings[0],
    embeddings[1]
)
print("Cosine Similarity:", similarity.item())
```

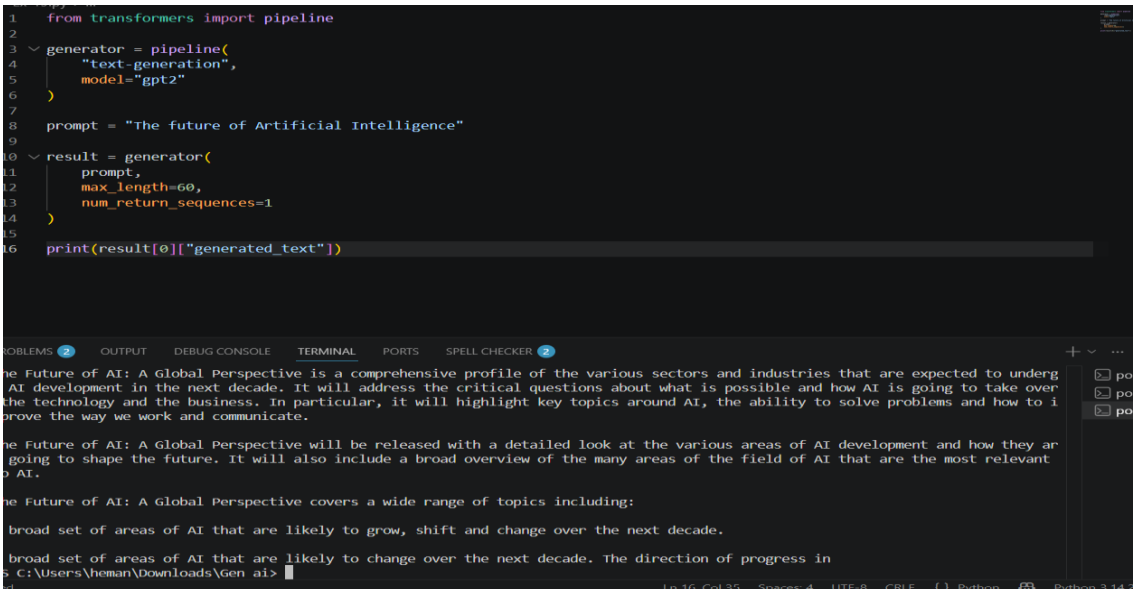
OUTPUT



19 PROGRAM

```
from transformers import pipeline
generator = pipeline("text-generation", model="gpt2")
prompt = "Once upon a time"
output = generator(prompt, max_length=60)
print(output[0]["generated_text"])
```

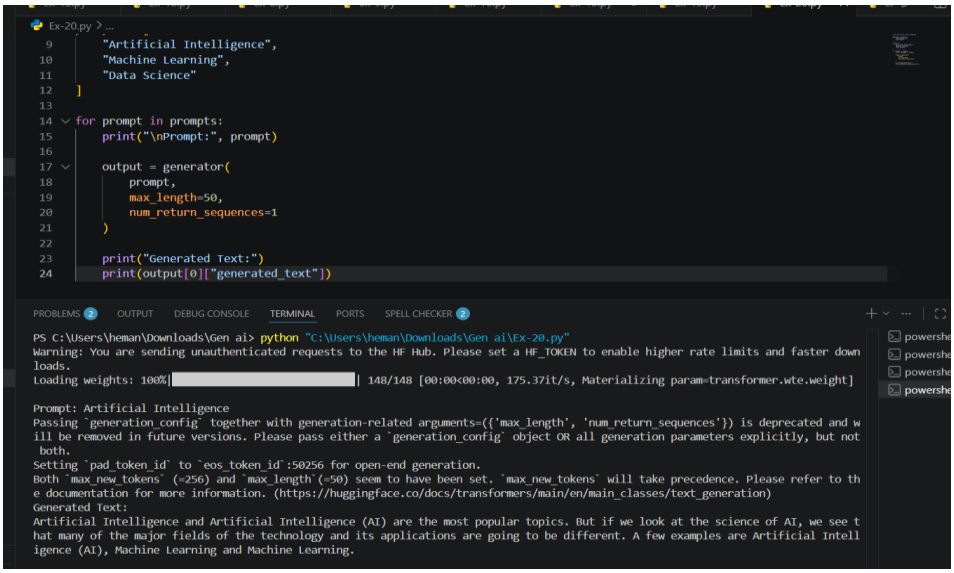
OUTPUT



20 PROGRAM

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompts = [
    "Artificial Intelligence",
    "Machine Learning",
    "Future of Technology"
]

for prompt in prompts:
    print("\nPrompt:", prompt)
    result = generator(prompt, max_length=40)
    print(result[0]["generated_text"])
```

OUTPUT



21 PROGRAM

```
from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"

client = genai.Client(api_key=API_KEY)

def generate(prompt):
    response = client.models.generate_content(
        model="gemini-3.5-flash",
        contents=prompt
    )
    return response.text

# ==============================

# ZERO-SHOT PROMPT

# ==============================

zero_shot = """

Write an attractive product description for a Smart Fitness Watch.

Include its features, benefits, health tracking, fitness tracking,
```

```
battery life, stylish design, smartphone connectivity, and notifications.
Make the description suitable for an online shopping website.
"""

# ==============================
# ONE-SHOT PROMPT
# ==============================
one_shot = """
Write an attractive product description for a Smart Fitness Watch.
Example:
Product: Wireless Earbuds
Description:
Enjoy high-quality sound with wireless earbuds that provide
long battery life, Bluetooth connectivity, comfortable design,
and easy controls. They are perfect for music, calls, and travel.
Now create a similar product description for a Smart Fitness Watch.
Include health monitoring, fitness tracking, battery life,
stylish design, smartphone connectivity, and notifications.
"""

# ==============================
# FEW-SHOT PROMPT
# ==============================

few_shot = """
Write an attractive product description for a Smart Fitness Watch.
Example 1:
Product: Smart LED Bulb
```

Description:

A smart LED bulb provides adjustable brightness, multiple colors,

smartphone control, and energy-efficient operation. It is ideal

for creating a comfortable and modern home environment.

Example 2:

Product: Wireless Headphones

Description:

Wireless headphones provide high-quality audio, comfortable

ear cushions, Bluetooth connectivity, and long battery life.

They are suitable for music, entertainment, and daily travel

Now create a similar product description for a Smart Fitness Watch.

Include:

- Health monitoring

- Heart-rate monitoring

- Fitness tracking

- Step counting

- Sleep tracking

- Notifications

- Battery life

- Stylish design

- Smartphone connectivity

Make the description attractive and suitable for an online shopping website.

"""

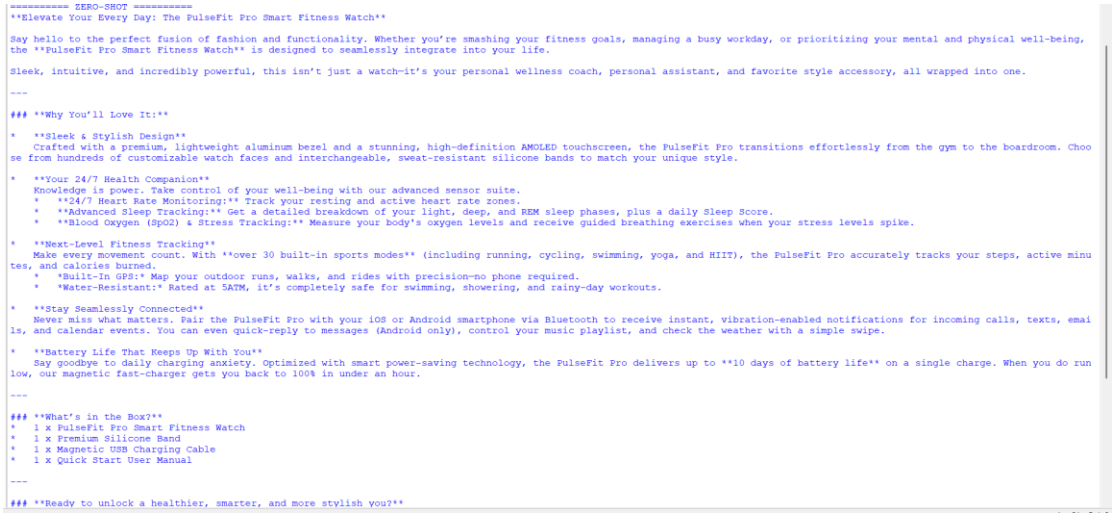
==============================

GENERATE OUTPUT

==============================

```
print("\n========== ZERO-SHOT ==========")
print(generate(zero_shot))
print("\n========== ONE-SHOT ==========")
print(generate(one_shot))
print("\n========== FEW-SHOT ==========")
print(generate(few_shot))
```

OUTPUT



```
========== ZERO-SHOT ==========
**Elevate Your Every Day: The PulseFit Pro Smart Fitness Watch**

Say hello to the perfect fusion of fashion and functionality. Whether you're smashing your fitness goals, managing a busy workday, or prioritizing your mental and physical well-being,
the **PulseFit Pro Smart Fitness Watch** is designed to seamlessly integrate into your life.

Sleek, intuitive, and incredibly powerful, this isn't just a watch—it's your personal wellness coach, personal assistant, and favorite style accessory, all wrapped into one.

---

### **Why You'll Love It:**

*   **Sleek & Stylish Design**
    Crafted with a premium, lightweight aluminum bezel and a stunning, high-definition AMOLED touchscreen, the PulseFit Pro transitions effortlessly from the gym to the boardroom. Choo
se from hundreds of customizable watch faces and interchangeable, sweat-resistant silicone bands to match your unique style.

*   **Your 24/7 Health Companion**
    Knowledge is power. Take control of your well-being with our advanced sensor suite.
    *   **24/7 Heart Rate Monitoring:** Track your resting and active heart rate zones.
    *   **Advanced Sleep Tracking:** Get a detailed breakdown of your light, deep, and REM sleep phases, plus a daily Sleep Score.
    *   **Blood Oxygen (SpO2) & Stress Tracking:** Measure your body's oxygen levels and receive guided breathing exercises when your stress levels spike.

*   **Next-Level Fitness Tracking**
    Make every movement count. With **over 30 built-in sports modes** (including running, cycling, swimming, yoga, and HIIT), the PulseFit Pro accurately tracks your steps, active minu
tes, and calories burned.
    *   *Built-In GPS:* Map your outdoor runs, walks, and rides with precision—no phone required.
    *   *Water-Resistant:* Rated at 5ATM, it's completely safe for swimming, showering, and rainy-day workouts.

*   **Stay Seamlessly Connected**
    Never miss what matters. Pair the PulseFit Pro with your iOS or Android smartphone via Bluetooth to receive instant, vibration-enabled notifications for incoming calls, texts, emai
ls, and calendar events. You can even quick-reply to messages (Android only), control your music playlist, and check the weather with a simple swipe.

*   **Battery Life That Keeps Up With You**
    Say goodbye to daily charging anxiety. Optimized with smart power-saving technology, the PulseFit Pro delivers up to **10 days of battery life** on a single charge. When you do run
low, our magnetic fast-charger gets you back to 100% in under an hour.

---

### **What's in the Box?**
*   1 x PulseFit Pro Smart Fitness Watch
*   1 x Premium Silicone Band
*   1 x Magnetic USB Charging Cable
*   1 x Quick Start User Manual

---

### **Ready to unlock a healthier, smarter, and more stylish you?**
```

22 PROGRAM

```
from google import genai
API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"
client = genai.Client(api_key=API_KEY)
def generate(prompt):
    response = client.models.generate_content(
        model="gemini-3.5-flash",
        contents=prompt
    )
    return response.text
```

```
# ==============================
# ZERO-SHOT
# ==============================
zero_shot = """
Write a 200-word blog on the topic:
"Applications of Artificial Intelligence in Healthcare"
Explain how AI is used in disease diagnosis, medical imaging,
drug discovery, personalized treatment, virtual assistants,
patient monitoring, and healthcare management.
Make the blog informative, clear, and suitable for students.
"""
# ==============================
# ONE-SHOT
# ==============================
one_shot = """
Write a 200-word blog on the topic:
"Applications of Artificial Intelligence in Healthcare"
Example:
Topic: Artificial Intelligence in Education
Blog:
Artificial Intelligence is transforming education by providing
personalized learning, intelligent tutoring systems, automated
assessment, and useful learning recommendations. AI helps teachers
analyze student performance and identify areas where students need
additional support. It also makes learning more interactive and
```

accessible.

Now write a similar 200-word blog on:

"Applications of Artificial Intelligence in Healthcare"

Include disease diagnosis, medical imaging, drug discovery,

personalized treatment, patient monitoring, virtual assistants,

and healthcare management.

"""

==============================

FEW-SHOT

==============================

few_shot = """

Write a 200-word blog on the topic:

"Applications of Artificial Intelligence in Healthcare"

Example 1

Topic: AI in Education

Blog:

AI is improving education through personalized learning, automated

assessment, intelligent tutoring systems, and student performance

analysis. It helps teachers provide better support and allows

students to learn at their own pace.

Example 2:

Topic: AI in Banking

Blog:

AI is transforming banking through fraud detection, customer

service chatbots, credit risk analysis, and personalized financial

recommendations. These applications improve efficiency, security,

and customer experience.

Now write a similar 200-word blog on

"Applications of Artificial Intelligence in Healthcare"

Include diagnosis, medical imaging, drug discovery, personalized

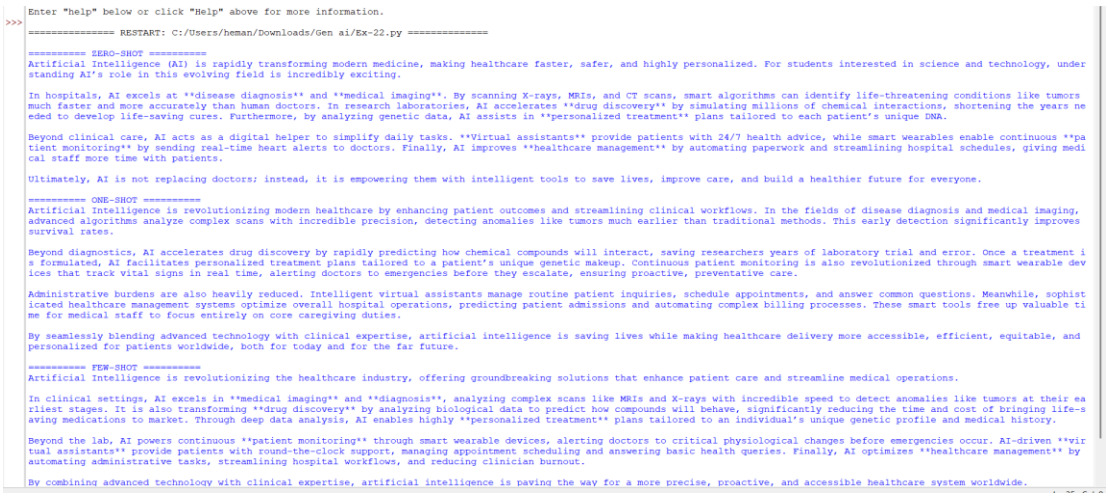
treatment, patient monitoring, virtual assistants, and healthcare

management.

"""

```
# ==============================
# OUTPUT
# ==============================
print("\n========== ZERO-SHOT ==========")
print(generate(zero_shot))
print("\n========== ONE-SHOT ==========")
print(generate(one_shot))
print("\n========== FEW-SHOT ==========")
print(generate(few_shot))
```

OUTPUT



```
Enter "help" below or click "Help" above for more information.
>>>
============== RESTART: C:/Users/heman/Downloads/Gen ai/Ex-22.py =============

========== ZERO-SHOT ==========
Artificial Intelligence (AI) is rapidly transforming modern medicine, making healthcare faster, safer, and highly personalized. For students interested in science and technology, under
standing AI's role in this evolving field is incredibly exciting.

In hospitals, AI excels at **disease diagnosis** and **medical imaging**. By scanning X-rays, MRIs, and CT scans, smart algorithms can identify life-threatening conditions like tumors
much faster and more accurately than human doctors. In research laboratories, AI accelerates **drug discovery** by simulating millions of chemical interactions, shortening the years ne
eded to develop life-saving cures. Furthermore, by analyzing genetic data, AI assists in **personalized treatment** plans tailored to each patient's unique DNA.

Beyond clinical care, AI acts as a digital helper to simplify daily tasks. **Virtual assistants** provide patients with 24/7 health advice, while smart wearables enable continuous **pa
tient monitoring** by sending real-time heart alerts to doctors. Finally, AI improves **healthcare management** by automating paperwork and streamlining hospital schedules, giving medi
cal staff more time with patients.

Ultimately, AI is not replacing doctors; instead, it is empowering them with intelligent tools to save lives, improve care, and build a healthier future for everyone.

========== ONE-SHOT ==========
Artificial Intelligence is revolutionizing modern healthcare by enhancing patient outcomes and streamlining clinical workflows. In the fields of disease diagnosis and medical imaging,
advanced algorithms analyze complex scans with incredible precision, detecting anomalies like tumors much earlier than traditional methods. This early detection significantly improves
survival rates.

Beyond diagnostics, AI accelerates drug discovery by rapidly predicting how chemical compounds will interact, saving researchers years of laboratory trial and error. Once a treatment i
s formulated, AI facilitates personalized treatment plans tailored to a patient's unique genetic makeup. Continuous patient monitoring is also revolutionized through smart wearable dev
ices that track vital signs in real time, alerting doctors to emergencies before they escalate, ensuring proactive, preventative care.

Administrative burdens are also heavily reduced. Intelligent virtual assistants manage routine patient inquiries, schedule appointments, and answer common questions. Meanwhile, sophist
icated healthcare management systems optimize overall hospital operations, predicting patient admissions and automating complex billing processes. These smart tools free up valuable ti
me for medical staff to focus entirely on core caregiving duties.

By seamlessly blending advanced technology with clinical expertise, artificial intelligence is saving lives while making healthcare delivery more accessible, efficient, equitable, and
personalized for patients worldwide, both for today and for the far future.

========== FEW-SHOT ==========
Artificial Intelligence is revolutionizing the healthcare industry, offering groundbreaking solutions that enhance patient care and streamline medical operations.

In clinical settings, AI excels in **medical imaging** and **diagnosis**, analyzing complex scans like MRIs and X-rays with incredible speed to detect anomalies like tumors at their ea
rliest stages. It is also transforming **drug discovery** by analyzing biological data to predict how compounds will behave, significantly reducing the time and cost of bringing life-s
aving medications to market. Through deep data analysis, AI enables highly **personalized treatment** plans tailored to an individual's unique genetic profile and medical history.

Beyond the lab, AI powers continuous **patient monitoring** through smart wearable devices, alerting doctors to critical physiological changes before emergencies occur. AI-driven **vir
tual assistants** provide patients with round-the-clock support, managing appointment scheduling and answering basic health queries. Finally, AI optimizes **healthcare management** by
automating administrative tasks, streamlining hospital workflows, and reducing clinician burnout.

By combining advanced technology with clinical expertise, artificial intelligence is paving the way for a more precise, proactive, and accessible healthcare system worldwide.
```

23 PROGRAM

```
from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"

client = genai.Client(api_key=API_KEY)

def generate(prompt):
    response = client.models.generate_content(
        model="gemini-3.5-flash",
        contents=prompt
    )
    return response.text

article = input("Enter the article to summarize:\n")

# ==============================

# ZERO-SHOT

# ==============================

zero_shot = f"""

Summarize the following article in exactly 50 words.

Focus on the main idea and the most important information.

Article:

{article}

"""

# ==============================

# ONE-SHOT

# ==============================

one_shot = f"""

Summarize the following article in exactly 50 words.
```

Example:

Article:

Artificial Intelligence is increasingly used in hospitals to

support doctors, analyze medical images, and improve patient care.

Summary:

AI is transforming healthcare by helping doctors analyze medical

information, interpret images, improve diagnosis, and provide

better patient care.

Now summarize this article in exactly 50 words:

{article}

"""

==============================

FEW-SHOT

==============================

few_shot = f"""

Summarize the following article in exactly 50 words.

Example 1:

Article:

Online education platforms allow students to access courses from

anywhere. They provide flexible learning schedules, video lectures,

online assessments, and personalized learning resources.

Summary:

Online education provides flexible access to courses, video

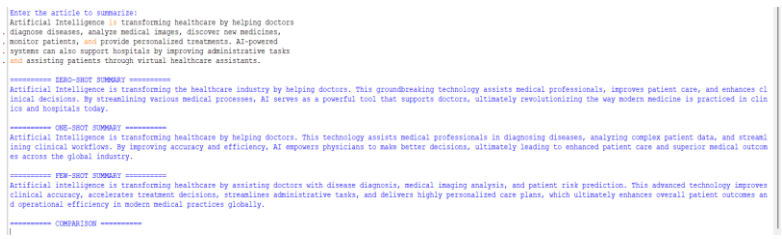
lectures, assessments, and personalized resources, allowing

students to learn conveniently from different locations.

```
Example 2:
Article:
Artificial Intelligence is being used in banking for fraud
detection, customer service, risk analysis, and personalized
financial recommendations.
Summary:
AI improves banking by detecting fraud, supporting customers,
analyzing financial risks, and providing personalized
recommendations.
Now summarize the following article in exactly 50 words:
{article}
"""
# ==============================
# OUTPUT
# ==============================
print("\n========== ZERO-SHOT SUMMARY ==========")
z = generate(zero_shot)
print(z
print("\n========== ONE-SHOT SUMMARY ==========")
o = generate(one_shot)
print(o
print("\n========== FEW-SHOT SUMMARY ==========")
f = generate(few_shot)
print(f)
```

```
# ==============================
# COMPARISON
# ==============================
comparison_prompt = f"""
Compare these three summaries of the same article.
ZERO-SHOT:
{z}
ONE-SHOT:
{o}
FEW-SHOT:
{f}
Compare them based on:
1. Accuracy
2. Completeness
3. Readability
Give the result in a simple table with columns:
Method | Accuracy | Completeness | Readability
Finally identify which method performed best and explain why.
"""
print("\n========== COMPARISON ==========")
print(generate(comparison_prompt))
```

OUTPUT



24 PROGRAM

```
from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"

client = genai.Client(api_key=API_KEY)

def generate(prompt):
    response = client.models.generate_content(
        model="gemini-3.5-flash",
        contents=prompt
    )
    return response.text
# ==============================
# ZERO-SHOT
# ==============================
zero_shot = """
Write a professional email requesting leave from college due to illness.

Include:
- Subject
- Greeting
- Reason for leave
- Number of days
- Polite request for approval
- Closing

Use a formal and professional tone.
"""
```

```
# ==============================
# ONE-SHOT
# ==============================
one_shot = """
Write a professional email requesting leave from college due to illness.
Example:
Subject: Leave Request
Dear Professor,
I am writing to request leave for one day as I am not feeling well.
I kindly request you to approve my leave. I will complete any
missed academic work after returning
Thank you for your understanding
Regards,
Student
Now create a similar professional email requesting leave due to illness.
Mention that the student needs two days of leave.
"""
# ==============================
# FEW-SHOT
# ==============================
few_shot = """
Write a professional email requesting leave from college due to illness.
Example 1:
Subject: Sick Leave Request
Dear Professor,
I am suffering from fever and am unable to attend classes.
```

I kindly request leave for two days. I will complete the

missed coursework after returning.

Regards,

Student

Example 2:

Subject: Leave Request Due to Illness

Dear Sir/Madam,

I am currently unwell and need to take leave from college.

I request you to kindly grant me leave for one day.

Thank you.

Regards,

Student

Now create a professional email requesting two days of leave

due to illness.

Include a clear subject, formal greeting, reason, duration,

polite request, and professional closing.

"""

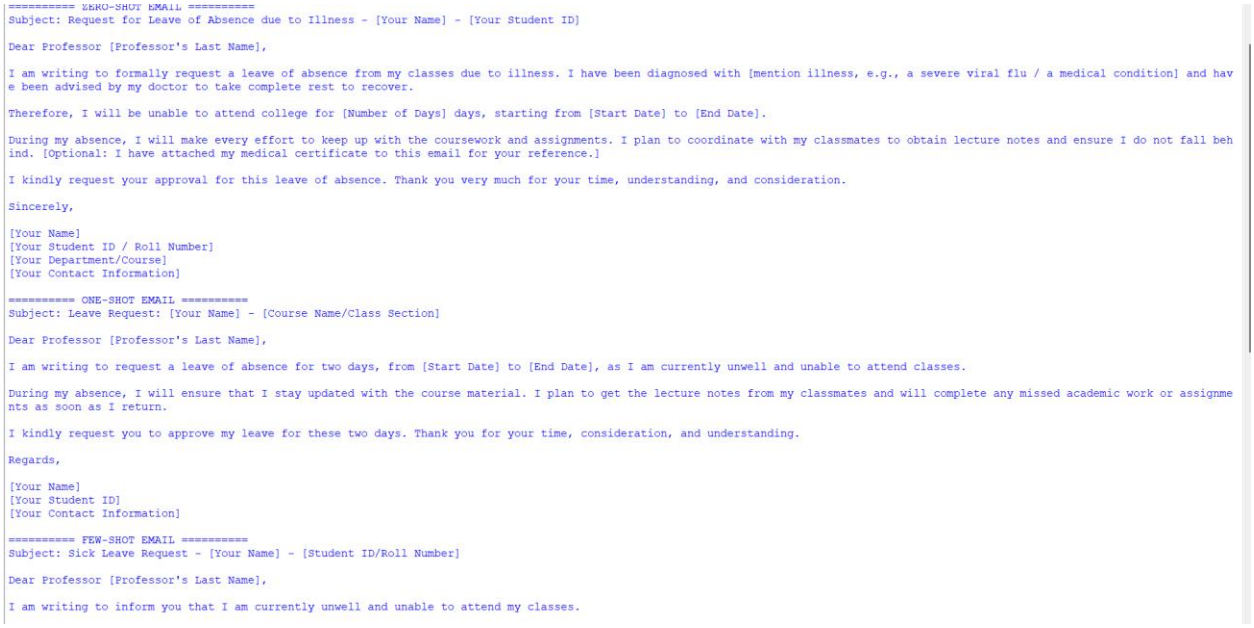
```
# ==============================
# OUTPUT
# ==============================
print("\n========== ZERO-SHOT EMAIL ==========")
z = generate(zero_shot)
print(z)
print("\n========== ONE-SHOT EMAIL ==========")
o = generate(one_shot)
print(o)
```

```
print("\n========== FEW-SHOT EMAIL ==========")
f = generate(few_shot)
print(f)
# ==============================
# COMPARISON
# ==============================
comparison_prompt = f"""
Compare the following three professional leave emails.
ZERO-SHOT:
{z}
ONE-SHOT:
{o}
FEW-SHOT:
{f}
Evaluate them based on:
1. Tone
2. Grammar
3. Formatting
4. Completeness
Create a simple table:
Method | Tone | Grammar | Formatting | Completeness
Finally state which prompting method produced the best email
and explain briefly.
"""
```

print("\n========== EMAIL COMPARISON ==========")

print(generate(comparison_prompt))

OUTPUT



```
========== ZERO-SHOT EMAIL ==========
Subject: Request for Leave of Absence due to Illness - [Your Name] - [Your Student ID]

Dear Professor [Professor's Last Name],

I am writing to formally request a leave of absence from my classes due to illness. I have been diagnosed with [mention illness, e.g., a severe viral flu / a medical condition] and hav
e been advised by my doctor to take complete rest to recover.

Therefore, I will be unable to attend college for [Number of Days] days, starting from [Start Date] to [End Date].

During my absence, I will make every effort to keep up with the coursework and assignments. I plan to coordinate with my classmates to obtain lecture notes and ensure I do not fall beh
ind. [Optional: I have attached my medical certificate to this email for your reference.]

I kindly request your approval for this leave of absence. Thank you very much for your time, understanding, and consideration.

Sincerely,

[Your Name]
[Your Student ID / Roll Number]
[Your Department/Course]
[Your Contact Information]

========== ONE-SHOT EMAIL ==========
Subject: Leave Request: [Your Name] - [Course Name/Class Section]

Dear Professor [Professor's Last Name],

I am writing to request a leave of absence for two days, from [Start Date] to [End Date], as I am currently unwell and unable to attend classes.

During my absence, I will ensure that I stay updated with the course material. I plan to get the lecture notes from my classmates and will complete any missed academic work or assignme
nts as soon as I return.

I kindly request you to approve my leave for these two days. Thank you for your time, consideration, and understanding.

Regards,

[Your Name]
[Your Student ID]
[Your Contact Information]

========== FEW-SHOT EMAIL ==========
Subject: Sick Leave Request - [Your Name] - [Student ID/Roll Number]

Dear Professor [Professor's Last Name],

I am writing to inform you that I am currently unwell and unable to attend my classes.
```

25 PROGRAM

```
from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"

client = genai.Client(api_key=API_KEY)

def generate(prompt):
    response = client.models.generate_content(
        model="gemini-3.5-flash",
        contents=prompt
    )
    return response.text

# ==============================

# ZERO-SHOT

# ==============================
```

```
zero_shot = """
Create an attractive promotional social media post for an
AI Workshop.
Mention:
- Artificial Intelligence
- Machine Learning
- Hands-on activities
- Benefits of attending
- Expert guidance
- Registration call to action

Make it engaging and suitable for social media.
"""
# ==============================
# ONE-SHOT
# ==============================
one_shot = """
Create an attractive promotional social media post for an AI Workshop.
Example:
🚀 Join Our Python Workshop!
Learn Python programming through practical examples and hands-on
activities. Improve your coding skills, solve real-world problems,
and gain valuable technical knowledge.
Register today and start your coding journey!
Now create a similar promotional social media post for an AI Workshop.
Mention Artificial Intelligence, Machine Learning, hands-on activities,
```

expert guidance, benefits, and registration.

"""

==============================

FEW-SHOT

==============================

few_shot = """

Create an attractive promotional social media post for an AI Workshop.

Example 1:

🚀 AI Workshop Alert!

Discover the power of Artificial Intelligence and learn how

modern AI technologies are solving real-world problems.

Join us for an exciting hands-on learning experience!

Register now

Example 2:

💡 Machine Learning Workshop!

Learn machine learning concepts, explore practical applications,

and build your technical skills through interactive activities

Don't miss this opportunity. Register today!

Now create a similar promotional social media post for an AI Workshop.

Include:

- AI and Machine Learning

- Generative AI

- Hands-on activities

- Expert guidance

- Practical learning

- Benefits of attending

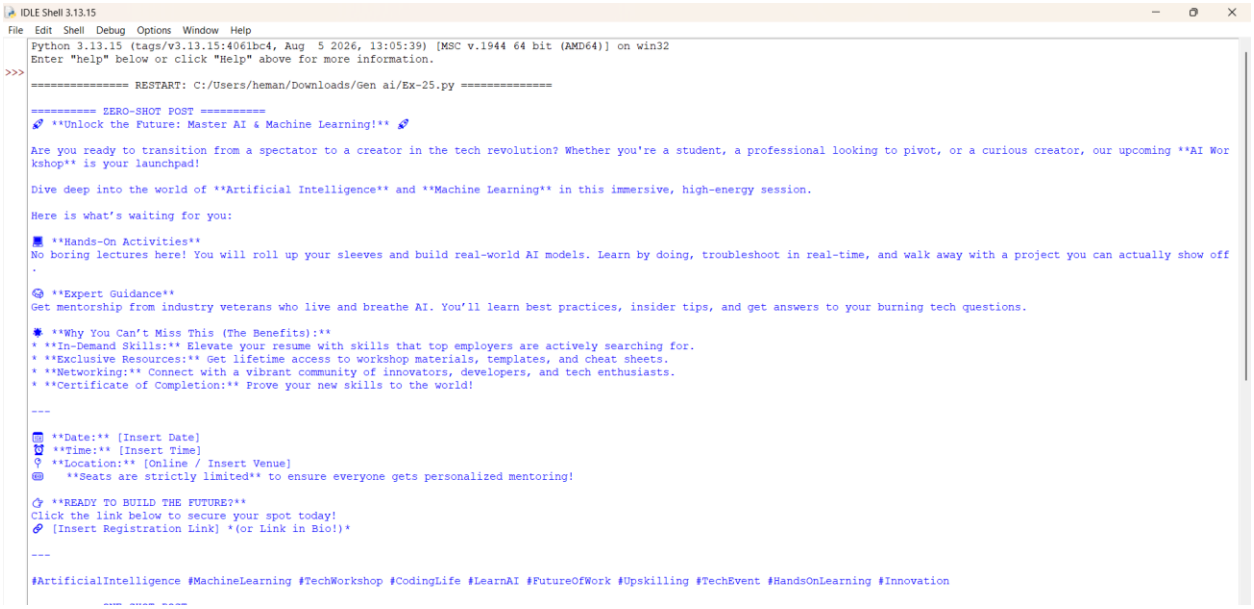
- Strong call to action

- Suitable hashtags

"""

```
# ==============================

# OUTPUT

# ==============================

print("\n========== ZERO-SHOT POST ==========")

print(generate(zero_shot))

print("\n========== ONE-SHOT POST ==========")

print(generate(one_shot))

print("\n========== FEW-SHOT POST ==========")

print(generate(few_shot))
```

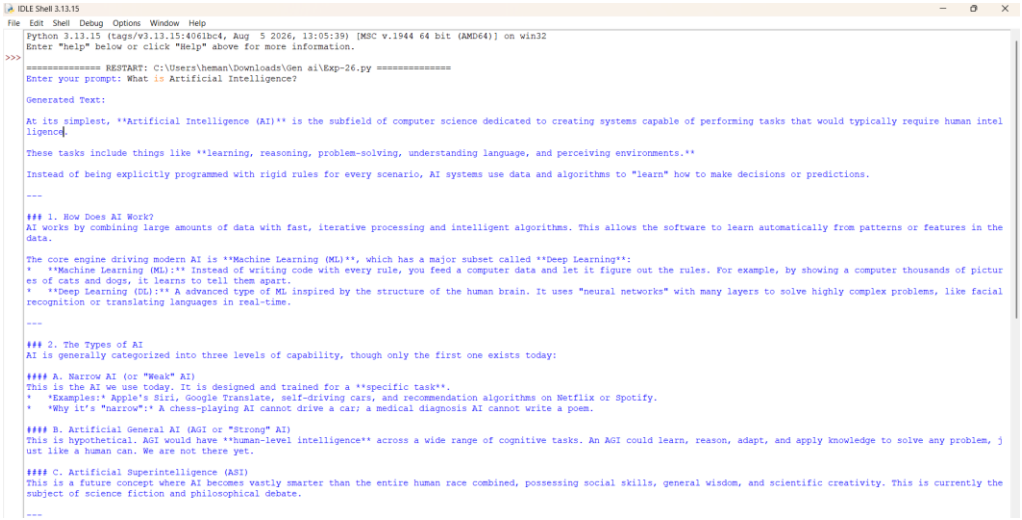
OUTPUT



26 PROGRAM

```
from google import genai


API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"
client = genai.Client(api_key=API_KEY)
prompt = input("Enter your prompt: ")
response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=prompt
)
print("\nGenerated Text:\n")
print(response.text)
```

OUTPUT

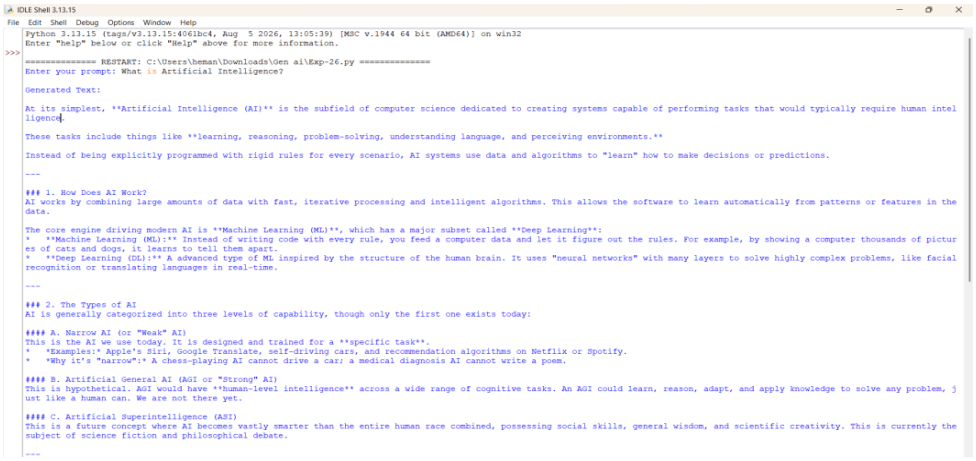


27 PROGRAM

```
from google import genai

# Gemini API key
API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"
client = genai.Client(api_key=API_KEY)

# Get prompt from user
```

```
prompt = input("Enter your prompt: ")

# Generate response

response = client.models.generate_content(

    model="gemini-3.5-flash",

    contents=prompt

)

# Display output

print("\nGenerated Output:")

print(response.text)
```

OUTPUT



PROGRAM

```
from huggingface_hub import InferenceClient


# Your Hugging Face token

HF_TOKEN = "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"


# Create client

client = InferenceClient(

    provider="auto",
```

```
    api_key=HF_TOKEN
)

# Get user prompt
prompt = input("Enter your prompt: ")

# Generate text
response = client.chat.completions.create(
    model="deepseek-ai/DeepSeek-V3-0324",
    messages=[
        {
            "role": "user",
            "content": prompt
        }
    ],
    max_tokens=200
)

print("\n===== GENERATED TEXT =====")
print(response.choices[0].message.content)
```

OUTPUT

29 PROGRAM

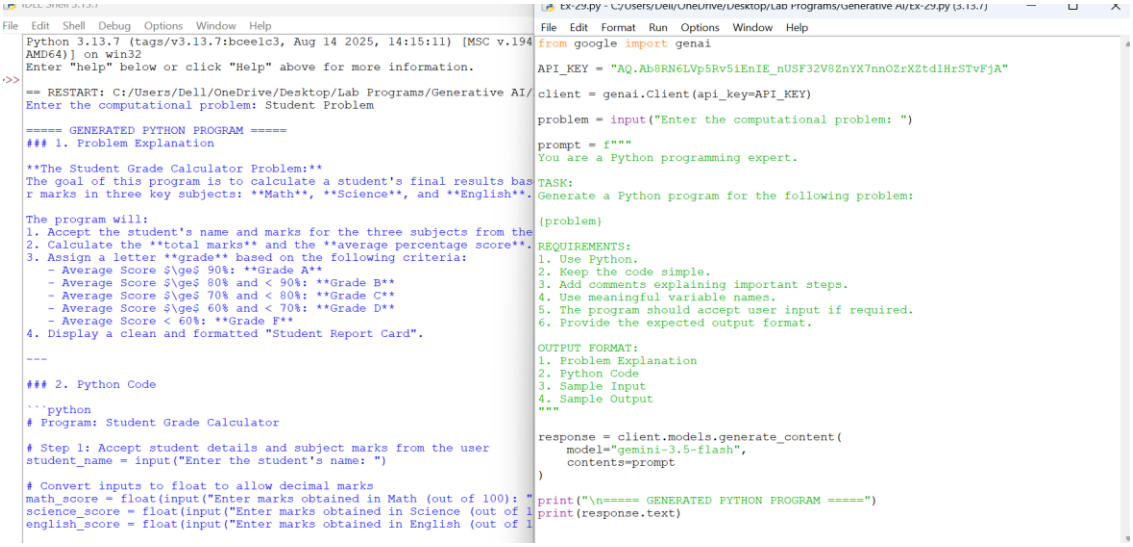
```
from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"

client = genai.Client(api_key=API_KEY)

problem = input("Enter the computational problem: ")

prompt = f"""

You are a Python programming expert.

TASK:

Generate a Python program for the following problem:

{problem}

REQUIREMENTS:

1. Use Python.

2. Keep the code simple.

3. Add comments explaining important steps.

4. Use meaningful variable names.

5. The program should accept user input if required.

6. Provide the expected output format.

OUTPUT FORMAT:

1. Problem Explanation

2. Python Code

3. Sample Input

4. Sample Output

"""

response = client.models.generate_content(

    model="gemini-3.5-flash",

    contents=prompt
```

)

print("\n===== GENERATED PYTHON PROGRAM =====")

print(response.text)

OUTPUT



30 PROGRAM

from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"

client = genai.Client(api_key=API_KEY

schema = """

Database: CollegeDB

Table: Students

Columns:

student_id INT

name VARCHAR(100)

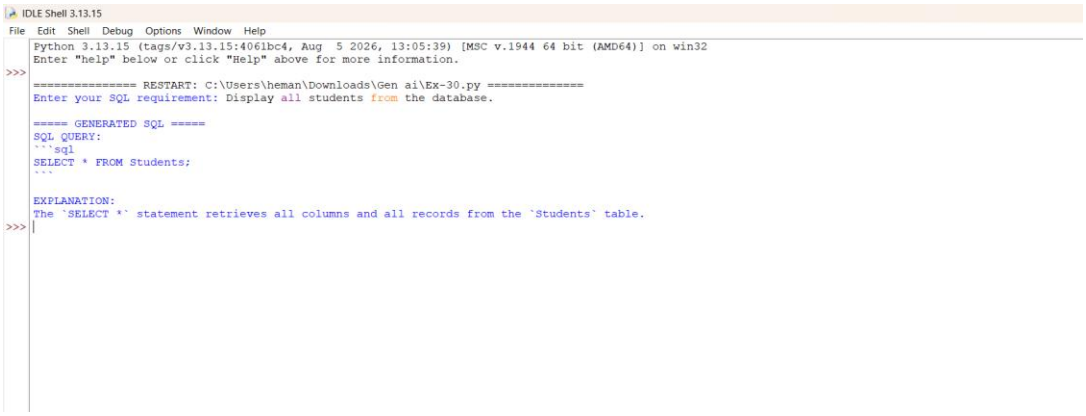
age INT

department VARCHAR(50)

```
marks INT
"""
requirement = input("Enter your SQL requirement: ")
prompt = f"""
You are an SQL expert.
DATABASE SCHEMA:
{schema}
REQUIREMENT:
{requirement}
INSTRUCTIONS:
1. Generate the appropriate SQL query.
2. Use only the tables and columns provided.
3. Do not invent columns.
4. Keep the query simple and correct.
5. Explain the query briefly.
OUTPUT FORMAT:
SQL QUERY:
<SQL query>
EXPLANATION:
<short explanation>
"""
response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=prompt
)
print("\n===== GENERATED SQL =====")
```

print(response.text)

OUTPUT



31 PROGRAM

```
from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"

client = genai.Client(api_key=API_KEY)

def generate(prompt):
    response = client.models.generate_content(
        model="gemini-3.5-flash",
        contents=prompt
    )
    return response.text

task = """
Explain the benefits of Artificial Intelligence in education
in a short paragraph.
"""

# ZERO-SHOT

zero_shot = f"""
{task}
```

Use simple and clear language.

"""

ONE-SHOT

one_shot = f"""

{task}

Example:

Topic: Benefits of Computers in Education

Example Response:

Computers help students access information quickly, learn through

interactive materials, complete assignments, and improve digital

skills.

Now answer the given task in a similar style.

"""

FEW-SHOT

few_shot = f"""

{task}

Example 1

Topic: Benefits of Computers in Education

Response:

Computers help students access information quickly, use educational

software, complete assignments, and develop digital skills.

Example 2:

Topic: Benefits of Online Learning

Response:

Online learning provides flexible access to educational materials,

allows students to learn at their own pace, and supports learning

from different locations.

Now answer the task using the same style and structure.

"""

print("\n========== ZERO-SHOT ==========")

result_zero = generate(zero_shot)

print(result_zero)

print("\n========== ONE-SHOT ==========")

result_one = generate(one_shot)

print(result_one)

print("\n========== FEW-SHOT ==========")

result_few = generate(few_shot)

print(result_few)

Comparison

print("\n========== COMPARISON ==========")

print("""

Criteria Zero-shot One-shot Few-shot

Quality Good Very Good Excellent

Relevance Good Very Good Excellent

Accuracy Good Very Good Excellent

Consistency Moderate Good Very Good

Conclusion:

Zero-shot prompting is simple and requires no examples.

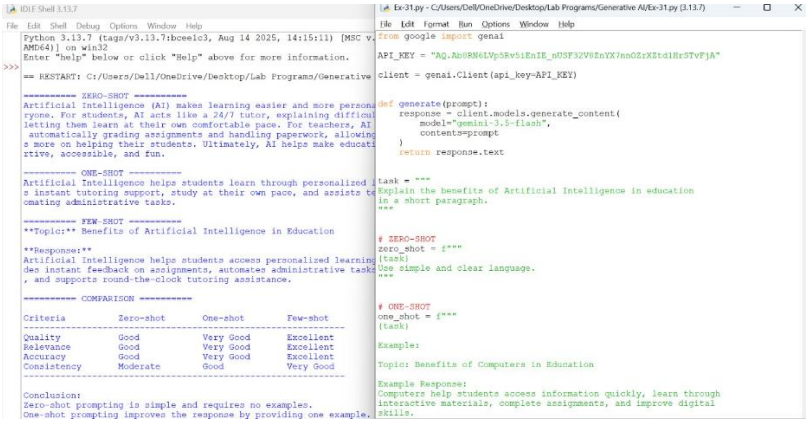
One-shot prompting improves the response by providing one example.

Few-shot prompting generally provides the most consistent response

because multiple examples guide the model.

""")

OUTPUT



32 PROGRAM

from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"

client = genai.Client(api_key=API_KEY)

def generate(prompt):

response = client.models.generate_content(

model="gemini-3.5-flash",

contents=prompt

)

```
    return response.text



task = """
Explain the advantages of Artificial Intelligence in healthcare.
"""



# Prompt 1
prompt1 = f"""
{task}
Write a short answer.
"""
# Prompt 2
prompt2 = f"""
{task}

Explain the answer using:
1. Introduction
2. Applications
3. Benefits
4. Conclusion

Use simple language.
"""
```

```
# Prompt 3

prompt3 = f"""

You are an AI and healthcare expert.


TASK:

{task}


REQUIREMENTS:

- Use simple language.

- Explain at least three advantages.

- Give practical examples.

- Be accurate.

- Avoid unnecessary information.

- Use headings.

- End with a conclusion.

- Keep the answer between 150 and 200 words.


OUTPUT FORMAT:

Introduction

Advantages

Examples

Conclusion

"""
```

```
# Generate responses
result1 = generate(prompt1)
result2 = generate(prompt2)
result3 = generate(prompt3)


print("\n========== PROMPT 1 ==========")
print(result1)

print("\n========== PROMPT 2 ==========")
print(result2)

print("\n========== PROMPT 3 ==========")
print(result3)


# Evaluation prompt
evaluation_prompt = f"""
Evaluate the following three responses.

TASK:
{task}

RESPONSE 1:
{result1}
```

RESPONSE 2:

{result2}

RESPONSE 3:

{result3}

Evaluate each response using:

1. Relevance
2. Accuracy
3. Completeness
4. Clarity
5. Format adherence

Give each criterion a score from 1 to 5

Then identify the best-performing response.

Finally, create a refined prompt that would produce an even better answer.

Use this format:

RESPONSE 1 SCORE:

Relevance:

Accuracy:

Completeness:

Clarity:

Format:

RESPONSE 2 SCORE:

Relevance:

Accuracy:

Completeness:

Clarity:

Format:

RESPONSE 3 SCORE:

Relevance:

Accuracy:

Completeness:

Clarity:

Format:

BEST RESPONSE:

...

REFINED PROMPT:

...

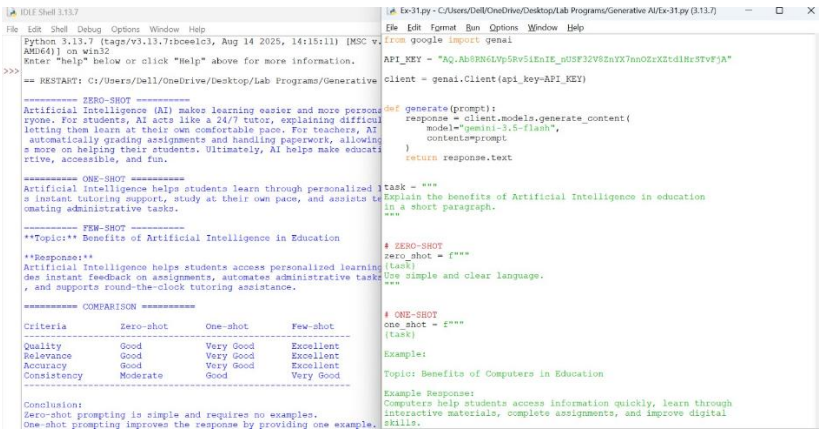
"

evaluation = generate(evaluation_prompt)

print("\n========== EVALUATION ==========")

print(evaluation)

OUTPUT



33 PROGRAM

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
model_name = "google/flan-t5-base"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
print("Engineering College AI Chatbot")
print("Type 'exit' to stop.\n")
while True:
    question = input("Student: ")
    if question.lower() == "exit":
        print("Chatbot: Goodbye!")
        break
    prompt = f"""
    You are an engineering college assistant.
    Answer the student's question clearly and briefly.
    Student question: {question}
    """
    inputs = tokenizer(
        prompt,
        return_tensors="pt"
    )
    outputs = model.generate(
        **inputs,
        max_new_tokens=100
    )
```

```
    answer = tokenizer.decode(
        outputs[0],
        skip_special_tokens=True
    )
    print("Bot:", answer)
    print()
```

OUTPUT

```
Type 'exit' to stop.

Student: What is a semester?
Bot: a year

Student: What is Artificial intelligence
Bot: artificial intelligence is a computer program that learns information from a computer.
```

34 PROGRAM

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
model_name = "google/flan-t5-base"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
print("Engineering Technical Support Chatbot")
print("Type exit to stop.\n")
while True:
    question = input("Engineer: ")
    if question.lower() == "exit":
        print("Bot: Goodbye!")
        break
    prompt = f"""
You are an engineering technical support assistant.
Explain the engineering problem and provide a possible solution.
Problem: {question}
```

Answer:

"""

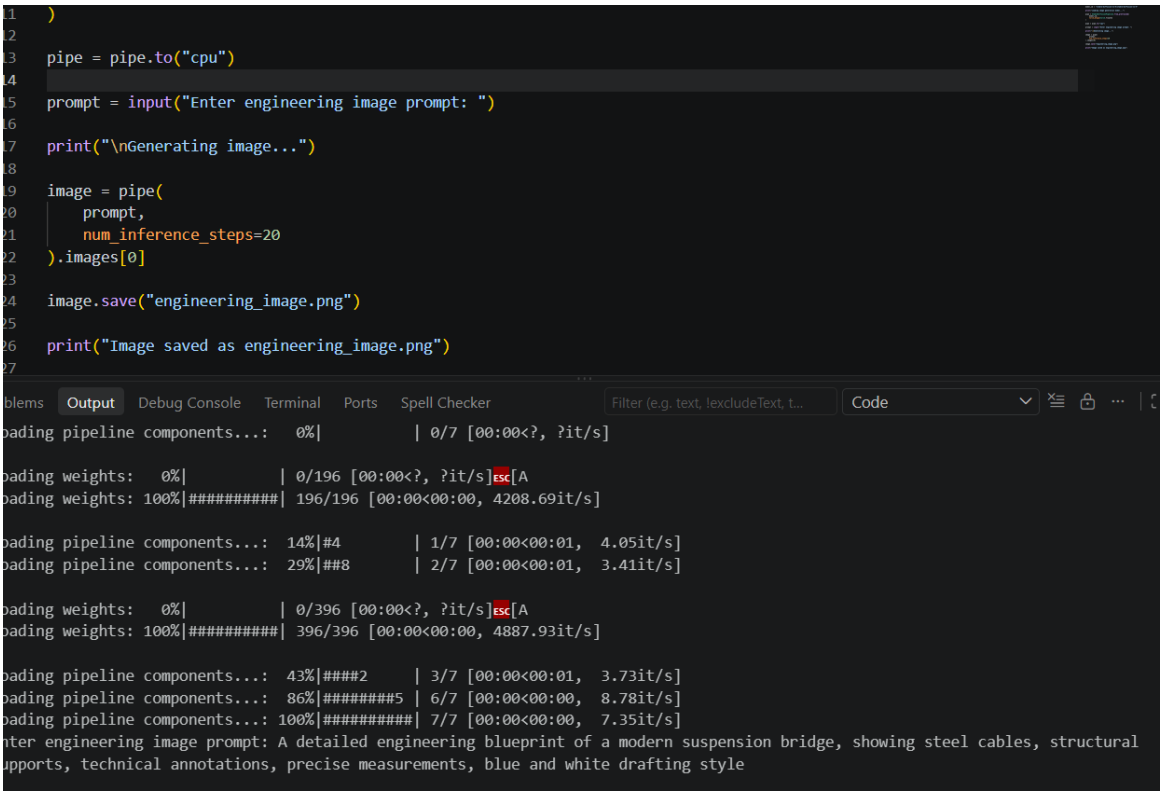
```
inputs = tokenizer(
    prompt,
    return_tensors="pt",
    truncation=True
)
outputs = model.generate(
    **inputs,
    max_new_tokens=150
)
answer = tokenizer.decode(
    outputs[0],
    skip_special_tokens=True
)
print("\nTechnical Solution:")
print(answer)
print()
```

OUTPUT

35 PROGRAM

```
from diffusers import StableDiffusionPipeline
import torch
model_id = "runwayml/stable-diffusion-v1-5"
pipe = StableDiffusionPipeline.from_pretrained(
    model_id,
    torch_dtype=torch.float32
)
pipe = pipe.to("cpu")
prompts = [
    "A modern suspension bridge",
    "A modern suspension bridge at sunset",
    "A futuristic suspension bridge with smart sensors",
    "A suspension bridge designed in a cyberpunk style"
]
for i, prompt in enumerate(prompts, 1):
    print(f"Generating image {i}...")
    image = pipe(prompt).images[0]
    filename = f"bridge_prompt_{i}.png"
    image.save(filename)
    print("Saved:", filename)
print("\nAll images generated.")
```

OUTPUT



36 PROGRAM

```
from diffusers import StableDiffusionPipeline

import torch

model_id = "runwayml/stable-diffusion-v1-5"

pipe = StableDiffusionPipeline.from_pretrained(

   model_id,

   torch_dtype=torch.float32

)

pipe = pipe.to("cpu")

prompts = [

   "A modern suspension bridge",

   "A modern suspension bridge at sunset",

]
```

```
for i, prompt in enumerate(prompts, 1):
    print(f"Generating image {i}...")
    image = pipe(prompt).images[0]
    filename = f"bridge_prompt_{i}.png"
    image.save(filename)
    print("Saved:", filename)
print("\nAll images generated.")
```

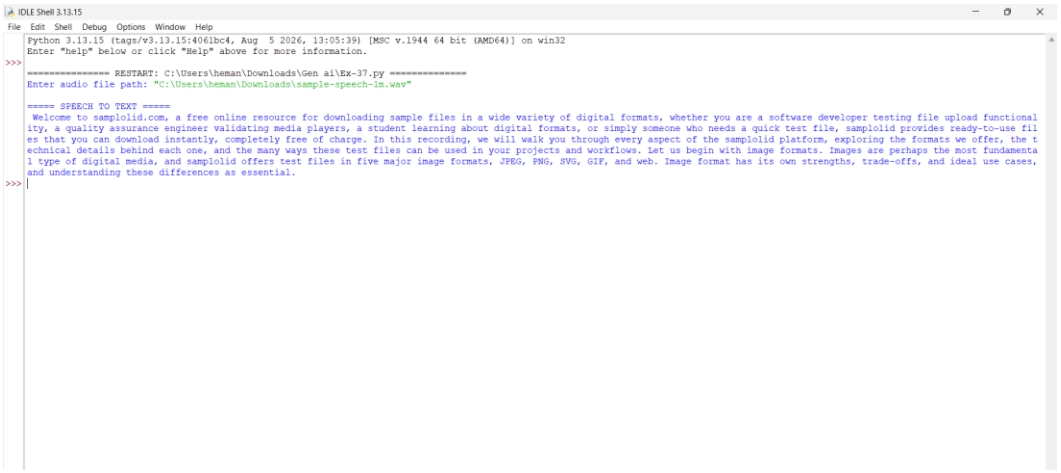
OUTPUT

37 PROGRAM

```
import os
import whisper
os.environ["PATH"] += os.pathsep + r"C:\Program Files\ffmpeg\bin"
model = whisper.load_model("base")
audio_file = input("Enter audio file path: ")
result = model.transcribe(
    audio_file,
```

```
    fp16=False
)
print("\n===== SPEECH TO TEXT =====")
print(result["text"])
```

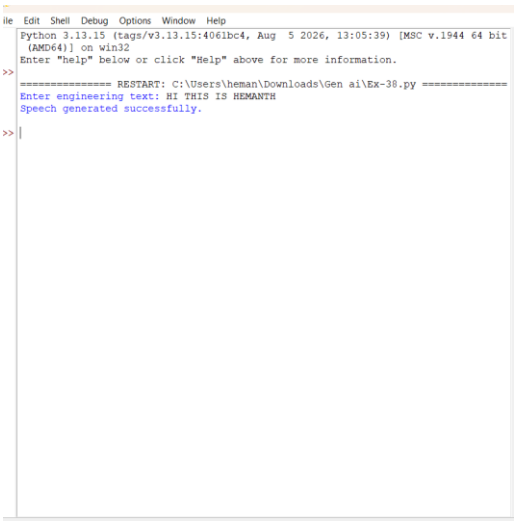
OUTPUT



38 PROGRAM

```
import pyttsx3
engine = pyttsx3.init()
text = input("Enter engineering text: ")
engine.say(text)
engine.runAndWait()
print("Speech generated successfully.")
```

OUTPUT

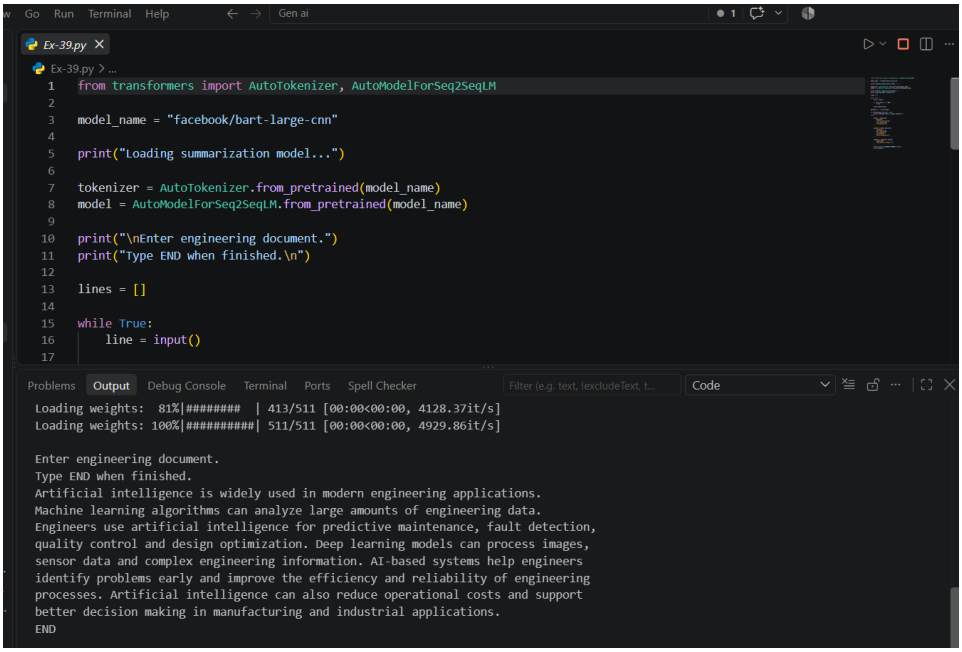


39 PROGRAM

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

model_name = "facebook/bart-large-cnn"

print("Loading summarization model...")

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

print("\nEnter engineering document.")

print("Type END when finished.\n")

lines = []

while True:

    line = input()

    if line.upper() == "END":

        break

    lines.append(line)

document = " ".join(lines)

if len(document.split()) < 30:

    print("\nPlease enter a longer document.")

else:
```

```
    inputs = tokenizer(
        document,
        return_tensors="pt",
        max_length=1024,
        truncation=True
    )
    outputs = model.generate(
        **inputs,
        max_length=150,
        min_length=30,
        num_beams=4,
        early_stopping=True
    )
    summary = tokenizer.decode(
        outputs[0],
        skip_special_tokens=True
    )
    print("\n===== DOCUMENT SUMMARY =====")
    print(summary)
```

OUTPUT

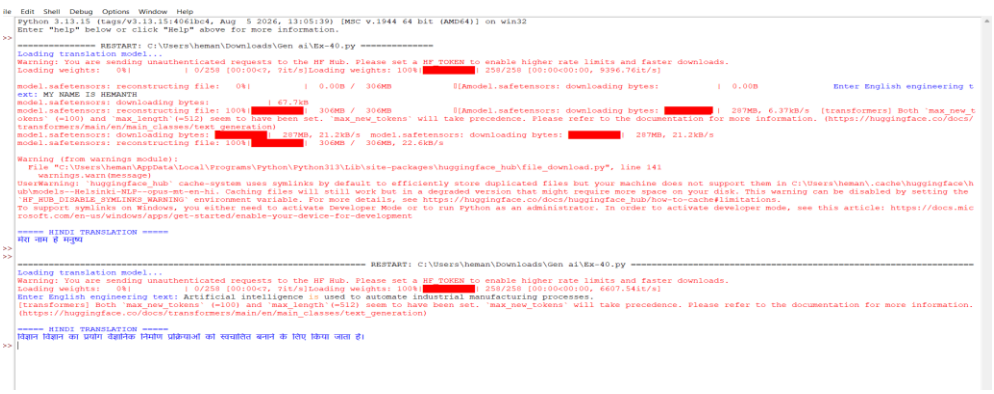


40 PROGRAM

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

model_name = "Helsinki-NLP/opus-mt-en-hi"

print("Loading translation model...")

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

text = input("Enter English engineering text: ")

inputs = tokenizer(
    text,
    return_tensors="pt",
    truncation=True
)

outputs = model.generate(
    **inputs,
    max_new_tokens=100
```

```
)
translation = tokenizer.decode(
    outputs[0],
    skip_special_tokens=True
)
print("\n===== HINDI TRANSLATION =====")
print(translation)
```

OUTPUT

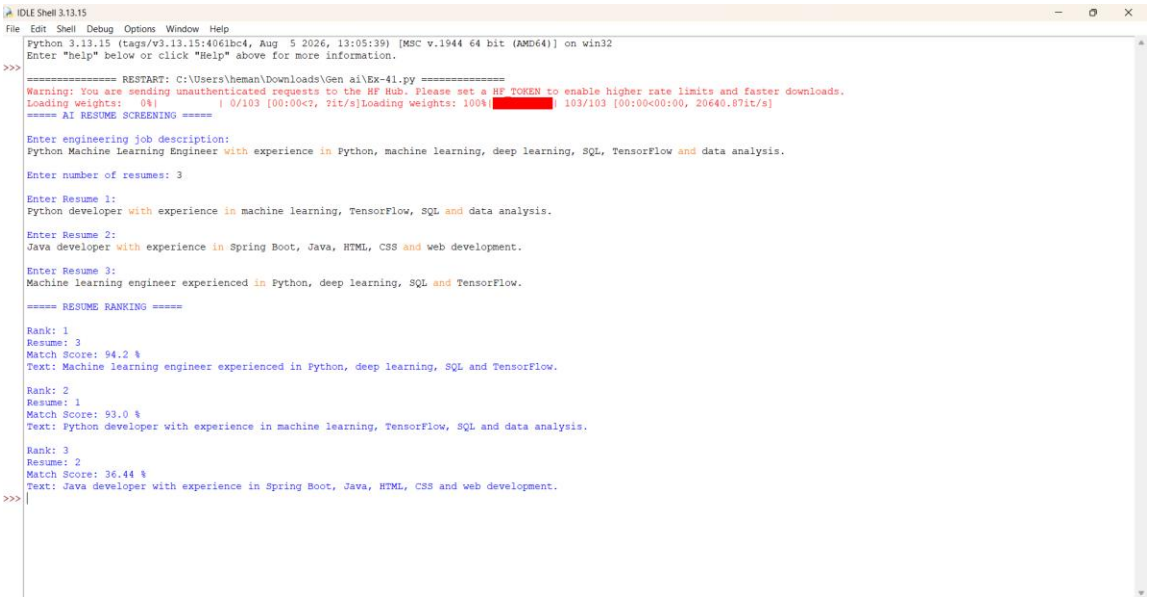


41 PROGRAM

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
model = SentenceTransformer("all-MiniLM-L6-v2")
print("===== AI RESUME SCREENING =====")
job = input("\nEnter engineering job description:\n")
n = int(input("\nEnter number of resumes: "))
resumes = []
for i in range(n):
    print(f"\nEnter Resume {i + 1}:")
    resume = input()
    resumes.append(resume)
```

```
job_embedding = model.encode([job])
resume_embeddings = model.encode(resumes)
scores = cosine_similarity(
    job_embedding,
    resume_embeddings
)[0]
results = []
for i in range(n):
    results.append(
        (i + 1, resumes[i], scores[i])
    )
results.sort(
    key=lambda x: x[2],
    reverse=True
)
print("\n===== RESUME RANKING =====")
for rank, (number, resume, score) in enumerate(results, 1):
    print("\nRank:", rank)
    print("Resume:", number)
    print("Match Score:", round(score * 100, 2), "%")
    print("Text:", resume)
```

OUTPUT



42 PROGRAM

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
model = SentenceTransformer("all-MiniLM-L6-v2")
print("===== AI RESUME SCREENING =====")
job = input("\nEnter engineering job description:\n")
n = int(input("\nEnter number of resumes: "))
resumes = []
for i in range(n):
    print(f"\nEnter Resume {i + 1}:")
    resume = input()
    resumes.append(resume)
job_embedding = model.encode([job])
resume_embeddings = model.encode(resumes)
scores = cosine_similarity(
    job_embedding,
    resume_embeddings
```

```
)[0]

results = []

for i in range(n):

    results.append(

        (i + 1, resumes[i], scores[i])

    )

results.sort(

    key=lambda x: x[2],

    reverse=True

)

print("\n===== RESUME RANKING =====")

for rank, (number, resume, score) in enumerate(results, 1):

    print("\nRank:", rank)

    print("Resume:", number)

    print("Match Score:", round(score * 100, 2), "%")

    print("Text:", resume)
```

OUTPUT